

## Script Contract v28

### BTNS-Native Polynomial Coefficient Construction Package

**BTNS-Native Polynomial Coefficient Construction Package with, Closed-Boundary Contract with Full Strict WFC, WFC Ablation, Independent Certified and Non-Certified Paths, Adaptive Certified Rebuild, and Matched-Depth Fairness Control**

---

#### 0. Contract Identity

**Contract title:**

Script Contract v28— BTNS-Native Polynomial Coefficient Construction Package with Full Strict Windowed Fractional Canonization, WFC Ablation, Independent Certified Path, Adaptive Rebuild, and Matched-Depth Fairness Control.

**Official package name:**

PythonApplication141 or later.

**Scientific reference label:**

PythonApplication\_BTNS\_Native\_v28\_FULL\_STRICT\_WFC\_MATCHED\_DEPTH

**Primary first-stage runner:**

run\_m2\_campaign.py

**Promotion runners:**

run\_m3\_campaign.py → run\_m4\_campaign.py → ... → run\_m10\_campaign.py

**Final full-campaign runner after staged validation:**

run\_m10\_campaign.py

**Initial executable degree:**

$m = 2$

**Promotion target:**

$m = 3, 4, 5, 6, 7, 8, 9, 10$

**Computational doctrine:**

BTNS-native internal coefficient construction with full strict WFC execution.

**WFC doctrine:**

The package shall execute the formal Windowed Fractional Canonization operator, not merely a generic re-normalization or trimming interface.

**Certified-path doctrine:**

Certified and Non-Certified shall share the same benchmark inputs, WFC settings, rounding settings, and root selections, but shall use distinct construction controllers.

**Fairness doctrine:**

Any claimed Certified improvement over fixed-depth Non-Certified shall be accompanied by matched-depth Non-Certified recomputation at the final depth selected by the Certified path.

The manifest shall record:

```
contract_version = v28
package_name = PythonApplication141
reference_label =
PythonApplication_BTNS_Native_v28_FULL_STRICT_WFC_MATCHED_DEPTH
wfc_execution_policy = FULL_STRICT_WFC_REQUIRED
wfc_ablation_policy = REQUIRED
matched_depth_fairness_policy = REQUIRED
```

---

## 1. Purpose of the Contract

This contract defines the mandatory implementation rules for a Python package whose purpose is to construct polynomial coefficients from prescribed sensitive roots using the BTNS System, the balanced ternary number system with digit set

$$D_3 = \{-1, 0, +1\} \quad \text{and radix } \beta = 3$$

The package shall demonstrate that coefficient construction is performed internally inside BTNS arithmetic using:

1. finite BTNSNumber digit objects;
2. carry-platform addition;
3. raw-convolution multiplication;
4. carry-platform normalization;
5. full strict Windowed Fractional Canonization;
6. UCC-governed Certified decisions;
7. adaptive Certified rebuild;
8. matched-depth Non-Certified fairness control;
9. reproducible workbook, CSV, JSON, trace, and manifest evidence.

For v28, the WFC requirement is strengthened beyond v22. The following is insufficient:

```
canonize_btns() as re-normalization only
implemented canonization interface only
WFC mentioned only as a formal paper definition
```

The required v28 behavior is:

```
formal WFC candidate enumeration
strict admissibility filtering
no-leakage checking
policy-selected canonical representative
WFC trace emission
WFC ablation reporting
```

Therefore, the package shall demonstrate not only native BTNS coefficient construction and Certified path separation, but also executable alignment with the formal WFC operator defined in the manuscript.

---

## 2. Governing Doctrine

The governing doctrine of v28 is:

Decimal arithmetic is allowed only at input ingress and output/report egress.

All coefficient-producing operations must be executed inside BTNS arithmetic.

Full strict WFC must be executed where canonization is required.

Excel workbooks are output artifacts only, never input dependencies.

Certified and Non-Certified shall share inputs, WFC settings, and benchmark settings.

Certified and Non-Certified shall not share the same construction controller.

Matched-depth Non-Certified recomputation is mandatory for fairness interpretation.

The valid comparison model is:

BTNS Non-Certified fixed-depth

versus

BTNS Certified adaptive-depth

versus

BTNS Non-Certified matched-depth control

The following models are forbidden:

Decimal Non-Certified versus BTNS Certified

Certified = Non-Certified + post-construction certificate only

WFC = ordinary normalize\_digits() only

WFC = canonize\_btns() without  $W_w$ ,  $c_{in}$ ,  $A_w^{strict}$ , and policy selection

Certified improvement reported without matched-depth fairness control

---

### 3. Scope of the Package

The package shall implement a staged campaign over ten sensitive-root groups.

The first executable stage shall use only two roots from each group:

$$m = 2$$

Only after the  $m = 2$  campaign passes all mandatory gates may the package be extended to:

$$m = 3$$

The full intended degree ladder is:

$$m = 2,3,4,5,6,7,8,9,10$$

No degree stage  $m > 2$  shall be treated as scientifically valid unless every preceding degree stage has satisfied the required gates for:

arithmetic correctness

BTNS legality

Certified/Non-Certified path separation

certified construction-time decision logic  
forbidden arithmetic audit  
traceability  
workbook output  
CSV/JSON sidecar output  
manifest reproducibility  
settings-signature reproducibility

### 3.1 Staged Execution Doctrine

The v28 package shall be developed and validated stage-by-stage. The first executable and manuscript-facing validation stage is  $m = 2$ . No  $m > 2$  result may be reported as a valid v28 result unless every preceding stage has passed all v28 gates, including arithmetic closure, full strict WFC execution, WFC trace completeness, WFC ablation generation, Certified/Non-Certified path separation, matched-depth fairness control, forbidden-arithmetic audit, sidecar generation, workbook generation, manifest reproducibility, and settings-signature reproducibility.

The existence of `run_m10_campaign.py` is permitted only after the  $m = 2$  through  $m = 9$  stages have passed promotion gates, or as a non-promotable development runner clearly labeled `DEVELOPMENT_ONLY`.

---

## 4. Input Root File Contract

The package shall accept benchmark root groups from a CSV file only.

The required file name for the first campaign is:

`sample_roots_m2.csv`

or a user-specified CSV path passed through the command line.

The required columns are:

`group_id, root_index, root_decimal_string, active_for_m, notes`

Rules:

1. `group_id` shall identify one of the sensitive-root groups.
2. `root_index` shall be a positive integer.
3. `root_decimal_string` shall be treated as ingress data only.
4. `active_for_m` may be true/false or an integer degree marker.
5. For  $m=2$ , the runner shall select `root_index = 1` and `root_index = 2` unless `active_for_m` explicitly overrides this selection.
6. No Excel workbook may be used as an input root source.

---

## 5. No External Excel Input Dependency Clause

The package shall not read any external Excel workbook as an input dependency for:

`roots`  
`coefficients`

gold values  
certified decisions  
benchmark summaries  
validation anchors  
historical references  
configuration

Excel workbooks are output artifacts only.

The only permitted benchmark inputs are:

1. CSV root input files
2. JSON configuration files
3. command-line parameters
4. Python constants used exclusively for unit tests

The benchmark runner and construction modules shall not call:

`openpyxl.load_workbook`  
`pandas.read_excel`  
`xlrd`  
`pyxlsb`  
any equivalent Excel-reading API

Any detected Excel-reading dependency shall be classified as:

`FORBIDDEN_EXCEL_INPUT`

and shall block campaign promotion.

---

## 6. Initial Mathematical Target: $m = 2$

For each sensitive-root group, the  $m = 2$  campaign shall use roots  $r_1$  and  $r_2$ . The constructed polynomial is:

$$P(x) = (x - r_1)(x - r_2)$$

Thus:

$$P(x) = x^2 - (r_1 + r_2)x + r_1r_2$$

The target non-leading coefficients are:

$$a_1 = -(r_1 + r_2), \quad a_2 = r_1r_2$$

The  $m = 2$  stage is mandatory because it isolates:

$$r_1 + r_2, \quad r_1r_2$$

If  $m = 2$  fails, no higher-degree stage may be accepted.

---

## 7. Definition of the Two Tested Computational Paths

## 7.1 BTNS Non-Certified Path

The Non-Certified path shall execute:

BTNS roots  $\rightarrow$  native BTNS recurrence controller  $\rightarrow$  BTNS coefficients

This path shall use legal BTNS primitives:

```
add_btns()  
sub_btns()  
neg_btns()  
mul_btns()  
normalize_digits()  
canonize_btns()
```

It shall not apply certified refinement, certified candidate selection, or construction-time certified acceptance/rejection.

Non-Certified does **not** mean decimal arithmetic.

## 7.2 BTNS Certified Path

The Certified path shall execute:

BTNS roots  $\rightarrow$  certified BTNS construction controller  $\rightarrow$  construction-time certificates  
 $\rightarrow$  certified candidate/refinement decisions  $\rightarrow$  BTNS Certified coefficients

Certified must not be implemented only as:

```
same recurrence  
different mode flag  
different source_tag  
different result class  
post-construction TRUE/TUBE/FALSE classification
```

Certified may produce final coefficients numerically equal to Non-Certified, but the construction trace must prove a distinct Certified construction process.

---

## 8. Certified/Non-Certified Fair-Input Equality Clause

The Certified and Non-Certified paths shall be evaluated under identical benchmark conditions.

For every group and degree stage, both paths shall receive the same:

$r_1, r_2, \dots, r_m,$   
 $k_{\min},$   
 $k_{\max},$   
rounding policy,  
CampaignConfig,  
input root selection

The purpose of the comparison is not to test different input conditions, but to test whether two distinct BTNS-native construction paths produce identical or different final coefficients under the same benchmark state.

Required:

For the fixed-depth comparison, Certified and Non-Certified shall receive the same initial  $k_{\min}$ ,  $k_{\max}$ , rounding policy, WFC policy, WFC window width, CampaignConfig, and root selection.

For the matched-depth comparison, Non-Certified shall be recomputed using the final  $k_{\min}$  selected by the Certified adaptive path, while preserving the same roots, ingress rounding policy, WFC policy, WFC window width,  $k_{\max}$  policy, support convention, tail convention, and CampaignConfig.

Any difference between outputs must originate from the internal construction logic of the two paths, not from different input data or different benchmark settings.

This preserves fair comparison while enforcing path separation.

---

## 9. Certified Construction Path Independence Clause

The Certified path shall not be implemented as a wrapper, relabeling layer, trace-tagging layer, or post-construction classification layer over coefficients already produced by the Non-Certified path.

The following pattern is forbidden:

```
noncert = build_polynomial_coeffs_noncertified(roots, config)
cert = build_polynomial_coeffs_certified(roots, config)

# forbidden if both internally call:
_build_polynomial_coefficients_native(...)
```

The required structure is:

```
noncert = build_polynomial_coeffs_noncertified(roots, config)
cert = build_polynomial_coeffs_certified_independent(roots, config)
```

where the Certified function executes a distinct certified construction algorithm, not merely the same recurrence with `certified=True`.

This directly addresses the defect identified in the Path Diagnosis, where both paths called the same internal builder and differed mainly by mode, certified flag, result type, and metadata.

---

## 10. No Shared Builder Clause

The package shall not allow the Certified and Non-Certified coefficient paths to use the same private builder function as their normative construction engine.

Forbidden design:

```
def build_polynomial_coeffs_noncertified(...):
    return _build_polynomial_coefficients_native(..., certified=False)
```

```
def build_polynomial_coeffs_certified(...):
    return _build_polynomial_coefficients_native(..., certified=True)
```

A shared low-level arithmetic kernel is permitted:

```
add_btns()
sub_btns()
neg_btns()
mul_btns()
normalize_digits()
canonize_btns()
```

However, a shared polynomial-construction controller is not permitted unless it is only a passive dispatcher and delegates to separate algorithms.

Required separation:

```
Non-Certified path:
    native recurrence controller
```

```
Certified path:
    certified construction controller
    operation-level certification
    interval/tail acceptance logic
    refinement/rebuild policy
    candidate acceptance or rejection
```

## 11. Certified Must Act During Construction Clause

Certified governance shall not be delayed until after final coefficients are already built.

The Certified path shall apply certified logic during coefficient construction at one or more of the following levels:

1. after each BTNS addition
2. after each BTNS multiplication
3. after each recurrence update
4. after each coefficient accumulation
5. before accepting a coefficient into the next polynomial stage

The following pattern is forbidden as the only Certified mechanism:

```
coefficients = build_coefficients(...)
certificates = [
    certify_coefficient(coefficient, gold_scaled[idx], policy)
    for idx, coefficient in enumerate(coefficients)
]
```

Post-construction certification may still be used for reporting, but it shall not be the only Certified mechanism.

The Certified path must include at least one construction-time decision point:

```
CERT_ACCEPT
CERT_REFINE
CERT_REBUILD
CERT_TUBE_HOLD
CERT_REJECT
```

---

## 12. Certified Refinement Loop Clause

The Certified path shall include an explicit refinement mechanism.

If a Certified operation or coefficient remains uncertain under the active precision, the path shall be allowed to request a deeper ingress projection and rebuild the affected coefficients.

The following behavior is insufficient as a final implementation:

```
if target_k_min < root.k_min:
    raise ValueError(REFINEMENT_REQUIRES_INGRESS)
```

Instead, v28 requires a formal refinement interface:

```
request_certified_refinement(
    root_records,
    current_k_min,
    target_k_min,
    rounding_policy,
    config
) -> refined_roots
```

The refinement itself may still occur through `ingress.py`, because decimal parsing belongs to the ingress boundary. However, the Certified path must be able to request and orchestrate refinement through the runner, rather than simply refusing deeper precision.

Required behavior:

1. Certified detects insufficient interval sharpness.
2. Certified emits `CERT_REFINE_REQUEST`.
3. Runner calls `ingress.py` to re-project the same root strings at deeper `k_min`.
4. Certified path rebuilds coefficients from refined `BTNSNumber` roots.
5. Final report records the refinement chain.

This closes the weakness that `refine_precision()` was only a placeholder and did not implement an adaptive certified loop.

---

## 13. Certified Candidate Selection Clause

The Certified path shall be permitted to generate and evaluate multiple BTNS-native candidate coefficients, provided all candidates are produced using BTNS operations only.

Candidate generation may include:

1. alternative local canonization choices
2. alternative carry-window decisions
3. deeper precision projections
4. interval-refined coefficient rebuilds
5. certified acceptance/rejection of candidate digits

The Certified path may select a candidate only by certified interval/tail logic, not by direct decimal/gold minimization.

**Forbidden:**

Choose the candidate with smallest decimal error against gold.

**Allowed:**

Choose the candidate whose BTNS interval satisfies the certified acceptance criterion.

Gold may be used only after construction for reporting and classification, not for selecting the Certified coefficient.

---

## 14. Certified Output Origin Clause

Every Certified coefficient shall carry metadata proving that it originated from the Certified construction path, not from copied Non-Certified coefficients.

Each Certified coefficient trace must include:

```
certified_path_id
certified_stage_id
certified_decision_chain
operation_certificate_ids
candidate_count
accepted_candidate_id
refinement_depth_used
```

The following is forbidden:

Certified coefficient trace differs from Non-Certified trace only by source\_tag, mode, or result class.

A Certified coefficient is invalid if its trace chain shows no Certified construction-time decision.

---

## 15. Certified/Non-Certified Trace Divergence Requirement

The package shall explicitly report whether the Certified and Non-Certified paths are:

```
VALUE_IDENTICAL_TRACE_DIFFERENT
VALUE_DIFFERENT_TRACE_DIFFERENT
VALUE_IDENTICAL_TRACE_IDENTICAL_INVALID
VALUE_DIFFERENT_TRACE_IDENTICAL_INVALID
```

A result where Certified and Non-Certified coefficients have identical numerical values is allowed.

However, a result where Certified and Non-Certified coefficients have identical construction traces is invalid.

Required interpretation:

Certified = Non-Certified in value:

acceptable only if construction traces prove distinct Certified construction logic.

Certified != Non-Certified in value:

acceptable if both paths are BTNS-native and Certified selection obeys certified logic.

Certified trace = Non-Certified trace:

invalid, because Certified has not been implemented as an independent path.

---

## 16. Certified Result Comparison Clause

The final comparison table shall distinguish between **value comparison** and **path comparison**.

For every coefficient, the report shall include:

```
coefficient_index
noncertified_digit_string
certified_digit_string
value_comparison_token
path_comparison_token
certified_decision_token
certified_construction_status
```

Allowed value comparison tokens:

```
CERT_VALUE_EQUAL_TO_NONCERT
CERT_VALUE_DIFFERENT_FROM_NONCERT
```

Allowed path comparison tokens:

```
CERT_PATH_DISTINCT_FROM_NONCERT
CERT_PATH_IDENTICAL_TO_NONCERT_INVALID
```

The IMPROVED/EQUAL/WORSE comparison remains an error-based comparison against external gold, but it is not sufficient to prove Certified-path independence.

Thus, v28 requires three independent comparisons:

1. numerical error comparison against gold
2. value equality/difference between Certified and Non-Certified
3. construction-path equality/difference between Certified and Non-Certified

---

## 17. Certified Path Failure Token Clause

If Certified does not execute an independent construction path, the run shall not be marked as `PROMOTABLE_RUN`, even if all coefficients are numerically accurate.

New failure token:

`CERTIFIED_PATH_NOT_DISTINCT`

The  $m = 2$  promotion gate shall include:

```
certified_path_distinct_count = coefficient_count
certified_path_not_distinct_count = 0
```

If not satisfied, the runner shall mark:

```
VALID_RUN
NOT_PROMOTABLE_RUN
```

This preserves diagnostic value while preventing scientifically invalid promotion.

---

## 18. External Gold Reference Contract

The package shall compute an external mathematical gold reference only for evaluation, reporting, and diagnostic comparison.

For roots:

$$r_1, r_2, \dots, r_m$$

the external gold polynomial is:

$$P_{\text{gold}}(x) = \prod_{i=1}^m (x - r_i)$$

The gold evaluator may use:

```
Decimal
Fraction
mpmath
high-precision rational arithmetic
```

but only inside:

```
gold_reference.py
diagnostics.py
reporting.py
```

The gold reference has exactly three permitted roles:

1. external truth comparison
2. error measurement
3. report generation

The gold reference has no permitted role in constructing BTNS coefficients or selecting Certified candidates.

Gold may classify results after construction, but it must not drive construction.

---

## 19. BTNSNumber and TailBound Representation Contract

All roots, intermediate values, partial products, coefficient accumulations, final coefficients, certified intervals, and trace values inside the construction path shall be represented as structured BTNS objects.

### 19.1 TailBound Object

```
@dataclass(frozen=True)
class TailBound:
    lower_exp: int
    abs_bound_num: int
    abs_bound_den_power3: int
    policy: str
    source: str
```

The absolute tail magnitude is interpreted as:

$$|T| \leq \frac{\text{abs\_bound\_num}}{3^{\text{abs\_bound\_den\_power3}}}$$

### 19.2 BTNSNumber Object

```
@dataclass(frozen=True)
class BTNSNumber:
    digits: dict[int, int]
    k_min: int
    k_max: int
    radix: int = 3
    digit_set: tuple[int, int, int] = (-1, 0, 1)
    tail_bound: TailBound | None = None
    canonicity_flag: str = "PENDING"
    source_tag: str = "UNSPECIFIED"
    precision_depth: int | None = None
    trace_id: str | None = None
```

The numerical interpretation is:

$$V(x) = \sum_{k=k_{\min}}^{k_{\max}} d_k 3^k$$

---

## 20.A. Rounding Policy Contract

Allowed rounding policies:

```
ROUND_TRUNCATE_TOWARD_ZERO
ROUND_NEAREST_TIES_TO_ZERO
ROUND_NEAREST_TIES_TO_EVEN
ROUND_FLOOR_BALANCED
```

The default policy for v28 shall be:

```
ROUND_NEAREST_TIES_TO_ZERO
```

No unregistered rounding policy is permitted.

Both Certified and Non-Certified paths must use the same ingress rounding policy for a given benchmark run.

## 20.B. WFC Policy Contract

The package shall define a WFC policy independently from the ingress rounding policy.

Allowed ingress rounding policies remain:

```
ROUND_TRUNCATE_TOWARD_ZERO
ROUND_NEAREST_TIES_TO_ZERO
ROUND_NEAREST_TIES_TO_EVEN
ROUND_FLOOR_BALANCED
```

Allowed WFC policies are:

```
RN_ZERO
RN_EVEN
```

where:

```
RN_ZERO corresponds to κ_ZERO
RN_EVEN corresponds to κ_EVEN
```

The WFC policy is not an external floating-point rounding mode. It is a deterministic selection policy over strictly valid finite BTNS digit candidates generated inside the boundary window.

Every run configuration shall record:

```
ingress_rounding_policy
wfc_policy
wfc_window_width
wfc_window_grid
wfc_ablation_enabled
```

Changing `wfc_policy` or `wfc_window_width` changes the semantic regime. Therefore, two runs with different WFC policies or window sizes shall not be treated as the same mathematical execution regime.

---

## 21. Internal Arithmetic Restrictions

Inside the coefficient-producing path, the following are permitted:

```
BTNSNumber
TailBound
```

balanced ternary digit vectors  
integer digit arithmetic on  $\{-1, 0, +1\}$   
carry propagation  
convolution over digit vectors  
BTNS interval/tail operations  
CampaignConfig  
structured trace/audit objects

**Inside the coefficient-producing path, the following are forbidden:**

float arithmetic  
Decimal arithmetic  
Fraction arithmetic  
NumPy polynomial construction  
NumPy root solving  
SymPy symbolic expansion  
direct decimal coefficient expansion  
direct use of external gold coefficients  
workbook-derived coefficient injection  
hidden conversion to decimal for arithmetic decisions  
Excel workbook reading  
egress conversion calls  
diagnostic comparator calls

**This restriction applies to:**

btns\_core.py  
btns\_poly.py  
btns\_certified.py

---

## 22. Construction Boundary Lock

**The construction modules:**

btns\_core.py  
btns\_poly.py  
btns\_certified.py

**shall not call:**

egress conversion functions  
decimal conversion functions  
gold-reference functions  
workbook readers  
diagnostic comparators  
btns\_to\_decimal\_string\_for\_report()  
to\_decimal()  
as\_decimal()  
any equivalent decimal-reporting function

**The only values allowed to cross into construction modules after ingress are:**

BTNSNumber objects  
TailBound objects  
CampaignConfig objects  
CertifiedPolicy objects  
structured trace/audit objects

---

## 23. Native BTNS Arithmetic Kernel

The module `btns_core.py` shall implement:

```
validate_digits()
normalize_digits()
add_btns()
neg_btns()
sub_btns()
mul_btns()
canonize_btns()
strict_wfc()
enumerate_wfc_candidates()
propagate_wfc_candidate()
filter_wfc_strict_candidates()
kappa_zero_key()
kappa_even_key()
select_wfc_candidate()
validate_wfc_trace()
btns_equal_digits()
```

The function `canonize_btns()` may remain as a compatibility wrapper only if it calls `strict_wfc()` or explicitly returns a trace token proving that WFC was not applicable under the declared operation context.

The following implementation pattern is forbidden:

```
def canonize_btns(x):
    return normalize_and_trim(x)
```

if the returned object is reported as WFC-canonical.

The following implementation pattern is required:

```
def canonize_btns(x, wfc_config, operation_context):
    return strict_wfc(x, wfc_config, operation_context)
```

or:

```
def canonize_btns(x, wfc_config, operation_context):
    result = strict_wfc_or_not_applicable(x, wfc_config,
operation_context)
    return result.canonical_object
```

where the returned trace records whether WFC was:

FULL\_STRICT\_WFC\_EXECUTED  
WFC\_NOT\_APPLICABLE  
WFC\_UNDEFINED\_STRICT\_EMPTY  
WFC\_TIE\_RESOLVED  
WFC\_REJECTED

### 23.1 Addition Law

$$s_k = x_k + y_k + c_{k-1}, \quad c_k = \left\lfloor \frac{s_k + 1}{3} \right\rfloor, \quad z_k = s_k - 3c_k$$

The output must satisfy:

$$z_k \in \{-1, 0, +1\}$$

### 23.2 Multiplication Law

$$u_k = \sum_i x_i y_{k-i}$$

Then:

$$s_k = u_k + q_{k-1}, \quad q_k = \left\lfloor \frac{s_k + 1}{3} \right\rfloor, \quad z_k = s_k - 3q_k$$

The final output must satisfy:

$$z_k \in \{-1, 0, +1\}$$

No decimal multiplication may replace this pathway.

### 23.3 Full Strict WFC Operator

For any legal BTNS digit object **z**, the strict WFC operator shall use the active boundary window

$$W_w = \{0, -1, -2, \dots, -w\}$$

The candidate carry-in set shall be exactly

$$C_{in} = \{-1, 0, +1\}$$

For each  $c_{in} \in C_{in}$ , the implementation shall generate a candidate by local carry propagation across the WFC window.

The implementation shall define the lower-boundary convention explicitly. Either:

LOWER\_BOUNDARY\_COMPENSATED

or

EXTENDED\_SUPPORT\_BOUNDARY

must be recorded. Silent value change due to an unaccounted incoming carry is forbidden.

For each candidate, the package shall compute:

$$s_k^{(c_{in})} = z_k + q_{k-1}^{(c_{in})}, \quad q_k^{(c_{in})} = \left\lfloor \frac{s_k^{(c_{in})} + 1}{3} \right\rfloor, \quad \tilde{z}_k^{(c_{in})} = s_k^{(c_{in})} - 3q_k^{(c_{in})}$$

For each  $c_{in} \in C_{in}$ , the implementation shall initialize

$$q_{-w-1}^{(c_{\text{in}})} = c_{\text{in}}$$

and shall propagate carries in increasing digit order:

$$k = -w, -w + 1, \dots, -1, 0$$

The emitted boundary carry  $q_0^{(c_{\text{in}})}$  shall be recorded and used for the strict no-leakage test.

The generated candidate family is:

$$\mathcal{A}_w(\mathbf{z}) = \{\tilde{\mathbf{z}}^{(c_{\text{in}})} : c_{\text{in}} \in \{-1, 0, +1\}\}$$

The strict candidate family is:

$$\mathcal{A}_w^{\text{strict}}(\mathbf{z}) = \{\tilde{\mathbf{z}}^{(c_{\text{in}})} \in \mathcal{A}_w(\mathbf{z}) : \mathcal{S}(\tilde{\mathbf{z}}^{(c_{\text{in}})}) = \text{TRUE}\}$$

where the strict predicate  $\mathcal{S}$  shall require all of the following:

1. digit legality:

$$\tilde{z}_k^{(c_{\text{in}})} \in \{-1, 0, +1\} \quad \forall k \in W_w$$

2. no leakage across the integer boundary:  $q_0^{(c_{\text{in}})} = 0$ ;
3. value preservation under the declared boundary convention:

$$\mathcal{V}_3(\tilde{\mathbf{z}}^{(c_{\text{in}})}) = \mathcal{V}_3(\mathbf{z})$$

or an explicitly recorded compensated-boundary equivalence;

4. no undeclared support expansion;
5. compatibility with the active tail/support convention;
6. deterministic traceability.

The canonical WFC output shall be:

$$\mathbf{z}^* = \text{Select}_P(\mathcal{A}_w^{\text{strict}}(\mathbf{z}))$$

where

$$P \in \{\text{RN\_ZERO}, \text{RN\_EVEN}\}$$

Equivalently,

$$\mathbf{z}^* = \arg \min_{\mathbf{e} \in \mathcal{A}_w^{\text{strict}}(\mathbf{z})} \kappa_P(\mathbf{e})$$

with deterministic secondary ordering if the policy key remains tied.

If  $\mathcal{A}_w^{\text{strict}}(\mathbf{z}) = \emptyset$ , the package shall not select a non-strict candidate. It shall return:

WFC\_UNDEFINED\_STRICT\_EMPTY

In the Certified path, this state may trigger:

CERT\_REFINE\_REQUEST  
CERT\_REBUILD

CERT\_TUBE\_HOLD  
CERT\_REJECT

according to UCC policy.

In the Non-Certified path, this state shall be reported as:

NONCERT\_WFC\_UNDEFINED

and shall block promotion unless the operation is explicitly classified as WFC\_NOT\_APPLICABLE.

Full strict WFC shall be invoked at every canonicalization boundary, including:

1. after any arithmetic operation whose result is committed as a BTNSNumber coefficient;
2. after any support truncation, support projection, or precision-depth reduction;
3. before accepting a coefficient into the next polynomial-construction stage;
4. before emitting any final Non-Certified, Certified, or matched-depth coefficient;
5. whenever `canonize_btns()` is called and the operation context is not explicitly classified as WFC\_NOT\_APPLICABLE.

If WFC is not invoked, the trace must state WFC\_NOT\_APPLICABLE and provide a deterministic reason.

## 23.4 WFC Data Structures

The package shall define explicit WFC data structures.

Mandatory data classes:

WFConfig  
WFCandidate  
WFCTrace  
WFCSelectionResult  
WFCAblationRecord

WFConfig shall include:

window\_width  
policy  
candidate\_C\_in\_set  
boundary\_mode  
strict\_value\_preservation\_required  
no\_leakage\_required  
secondary\_ordering

WFCandidate shall include:

candidate\_id  
operation\_id  
input\_trace\_id  
c\_in  
window\_width  
digits\_before  
digits\_after  
q0

legal\_digit\_status  
value\_preservation\_status  
support\_status  
no\_leakage\_status  
strict\_status  
policy\_key\_zero  
policy\_key\_even

WFCTrace shall include:

wfc\_trace\_id  
operation\_id  
input\_trace\_id  
output\_trace\_id  
window\_width  
policy  
candidate\_count  
strict\_candidate\_count  
selected\_candidate\_id  
selected\_c\_in  
q0\_selected  
wfc\_status  
wfc\_boundary\_mode  
wfc\_not\_applicable\_reason

---

## 24. Tail and Precision Boundary Rule

No digit contribution below active  $k_{\min}$  may be silently discarded.

For v28, this rule applies also to WFC candidate generation.

If WFC uses an incoming carry  $c_{\text{in}}$  at the lower boundary of  $W_w$ , then the implementation shall do one of the following:

1. represent the compensating lower-boundary effect explicitly;
2. expand support and propagate the effect through declared digit positions;
3. record the effect as a structured TailBound object;
4. reject the candidate as non-strict.

The following behavior is forbidden:

inject  $c_{\text{in}}$  into  $W_w$   
alter the boundary-window value  
ignore the missing compensating digit  
accept the candidate as strict

No carry above the window boundary may be silently accepted. The strict WFC no-leakage condition requires:

$$q_0 = 0$$

If  $q_0 \neq 0$ , the candidate is not strictly valid for WFC, even if its digits are legal.

---

## 25. Root Ingress Contract

The module `ingress.py` shall convert decimal root strings into BTNS objects.

Mandatory function:

```
parse_decimal_string_to_btns(  
    root_string: str,  
    k_min: int,  
    k_max: int | None,  
    rounding_policy: str,  
    trace_id: str  
) -> BTNSNumber
```

The ingress parser may use high-precision Decimal or Fraction arithmetic to interpret the input string and project it into a finite BTNS digit object.

However:

1. the resulting root object must be a BTNSNumber
2. all later coefficient-producing operations must use only the BTNSNumber
3. the original decimal root must not be used inside the construction engine
4. `k_max` may be inferred from integer magnitude at ingress
5. `k_min` is campaign-controlled
6. arithmetic may extend `k_max` dynamically when carry requires it
7. the parser must emit a complete ingress trace

Certified refinement requests must be handled by returning to this ingress boundary, not by allowing decimal parsing inside construction modules.

---

## 26. Egress and Reporting Contract

The module `egress.py` may convert BTNS outputs to decimal strings for reporting and human readability.

Mandatory function:

```
btns_to_decimal_string_for_report(  
    x: BTNSNumber,  
    digits: int  
) -> str
```

This conversion is egress-only.

The decimal result may be used in:

Excel workbooks  
CSV files  
JSON reports

diagnostic summaries  
manuscript tables

It shall not feed back into:

btns\_core.py  
btns\_poly.py  
btns\_certified.py

or any coefficient-producing computation.

---

## 27. Polynomial Construction Contract

The module `btns_poly.py` shall implement:

```
build_polynomial_coeffs_noncertified()  
build_polynomial_coeffs_certified_independent()  
build_polynomial_coeffs_noncertified_matched_depth()
```

The Non-Certified fixed-depth path shall execute:

```
BTNS roots  
→ native recurrence controller  
→ BTNS add/mul/sub primitives  
→ WFC gate where canonical acceptance is required  
→ BTNS coefficients
```

The Certified adaptive path shall execute:

```
BTNS roots  
→ independent certified construction controller  
→ BTNS add/mul/sub primitives  
→ full strict WFC candidate enumeration and selection where canonical  
acceptance is required  
→ UCC operation witness  
→ CERT_ACCEPT / CERT_REFINE / CERT_REBUILD / CERT_TUBE_HOLD / CERT_REJECT  
→ BTNS Certified coefficients
```

The matched-depth Non-Certified control shall execute:

```
same roots  
same final k_min selected by Certified  
same k_max policy  
same ingress rounding policy  
same WFC window width  
same WFC tie policy  
same CampaignConfig  
Non-Certified controller only
```

The following is forbidden:

Certified uses full WFC but Non-Certified bypasses WFC in the same main comparison.

Non-Certified matched-depth uses different WFC settings from Certified. Matched-depth comparison is inferred from existing fixed-depth results. Matched-depth comparison is skipped when Certified improves over fixed-depth Non-Certified.

---

## 28. Certified Governance Contract

The module `btncs_certified.py` shall implement Certified construction governance with WFC-aware certificates.

Mandatory functions include:

```
compute_tail_bound()
make_interval_enclosure()
certify_operation()
certify_coefficient()
classify_true_tube_false()
request_certified_refinement()
certified_candidate_acceptance()
compare_certified_path_to_noncertified_path()
compare_certified_path_to_matched_depth_noncertified_path()
certify_wfc_candidate()
certify_wfc_selection()
classify_wfc_status()
```

Every Certified addition, subtraction, negation, multiplication, WFC selection, and coefficient accumulation shall emit an operation-level certificate according to the mandatory schema defined in Clause 28.1.

Gold/reference values may not be used to select WFC candidates. WFC selection shall be performed only through:

```
strict admissibility
κ_ZERO or κ_EVEN
deterministic secondary ordering
UCC construction-time acceptance logic
```

### 28.1 Operation-Level Certificate

Every Certified addition, subtraction, negation, multiplication, WFC selection, and coefficient accumulation shall emit one operation-level certificate containing both arithmetic and WFC fields:

```
operation_id
operation_name
input_trace_ids
output_trace_id
k_min
k_max
tail_bound
canonicity_status
wfc_trace_id
```

```

wfc_status
wfc_window_width
wfc_policy
wfc_candidate_count
wfc_strict_candidate_count
wfc_selected_c_in
wfc_selected_candidate_id
decision_token
refinement_status
certified_construction_token

```

## 28.2 Coefficient-Level Certificate

Every final coefficient in the Certified path shall receive an interval:

$$I_{\text{cert}} = [c_L, c_U]$$

Let  $g$  be the external gold value used only for reporting and post-construction classification.

Decision law:

TRUE:

if  $g \in I_{\text{cert}}$  and  $\text{interval\_width}(I_{\text{cert}}) \leq \text{accepted\_width\_threshold}$ .

FALSE:

if  $g \notin I_{\text{cert}}$ .

TUBE:

if  $g \in I_{\text{cert}}$  but  $\text{interval\_width}(I_{\text{cert}}) > \text{accepted\_width\_threshold}$ ,  
or if the refinement limit is reached without sufficient enclosure sharpness.

Gold may participate only in **post-construction certificate classification**, not in construction or coefficient selection.

## 29.1. Precision-Depth Campaign Contract

Default grid:

```
K_MIN_GRID = [-20, -30, -40, -50]
```

For each depth, the package shall report:

```

Certified mean error
Non-Certified mean error
Certified max error
Non-Certified max error
TRUE count
TUBE count
FALSE count
IMPROVED count

```

EQUAL count  
WORSE count  
certified\_path\_distinct\_count  
certified\_path\_not\_distinct\_count  
certified\_construction\_decision\_count  
illegal digit count  
carry failure count  
trace completeness count  
forbidden arithmetic count

## 29.2. Matched-Depth Fairness Control Contract

Matched-depth fairness is mandatory in v28.

For every Certified coefficient produced at final depth  $k_{\min}^{\text{cert,final}}$ , the package shall recompute the corresponding Non-Certified coefficient using:

$$k_{\min}^{\text{noncert,matched}} = k_{\min}^{\text{cert,final}}$$

The matched-depth run shall use the same:

root group  
active roots  
root decimal strings at ingress  
ingress rounding policy  
k\_max policy  
WFC window width  
WFC tie policy  
CampaignConfig  
support convention  
tail convention

The matched-depth run shall not use:

Certified UCC decisions  
Certified candidate acceptance  
Certified refinement decisions  
Certified traces copied into Non-Certified  
external gold selection  
decimal coefficient construction

The matched-depth comparison shall report:

coefficient\_index  
group\_id  
initial\_k\_min  
certified\_final\_k\_min  
noncert\_fixed\_k\_min  
noncert\_matched\_k\_min  
certified\_digit\_string  
noncert\_fixed\_digit\_string  
noncert\_matched\_digit\_string

fixed\_depth\_comparison  
matched\_depth\_comparison  
adaptive\_advantage\_classification

**Allowed matched-depth comparison tokens:**

MATCHED\_DEPTH\_EQUAL  
MATCHED\_DEPTH\_DIFFERENT  
MATCHED\_DEPTH\_NOT\_RUN\_INVALID

**Allowed adaptive-advantage tokens:**

PRECISION\_ADVANTAGE\_ONLY  
NO\_ADAPTIVE\_REBUILD\_NEEDED  
CERT\_ADAPTIVE\_ADVANTAGE\_CONFIRMED  
CERT\_ADAPTIVE\_WORSE\_THAN\_MATCHED\_NONCERT  
MATCHED\_NONCERT\_REPRODUCES\_CERT  
MATCHED\_NONCERT\_DIFFERS\_FROM\_CERT

A run shall not claim Certified numerical superiority unless matched-depth evidence supports it.

If Certified improves over fixed-depth Non-Certified but matched-depth Non-Certified reproduces Certified, the required interpretation is:

Certified improvement is a precision-depth/adaptive-governance effect, not intrinsic equal-depth arithmetic superiority.

---

## 30. Unit-Test Gate Contract

Before any sensitive-root benchmark executes, the package shall run mandatory self-tests.

**Mandatory tests:**

1. digit legality test
2. addition closure test
3. multiplication closure test
4. additive identity test
5. multiplicative identity test
6. additive inverse / negation test
7. subtraction consistency test
8. addition commutativity test
9. multiplication commutativity test
10. distributivity spot-check
11. carry propagation stress test
12. fractional boundary test
13. conversion round-trip diagnostic test
14. known quadratic polynomial test
15. known cubic polynomial test
16. canonicity preservation test
17. forbidden arithmetic audit test
18. no external Excel input test

19. construction boundary lock test
20. tail-bound structure test
21. no silent truncation test
22. certified path independence test
23. certified construction-time decision test
24. no shared builder test
25. certified trace divergence test
26. WFC candidate enumeration test
27. WFC c\_in set test
28. WFC no-leakage q0 = 0 test
29. WFC strict-empty-family test
30. WFC value-preservation test
31. WFC lower-boundary compensation test
32. WFC support-expansion rejection test
33. WFC κ\_ZERO determinism test
34. WFC κ\_EVEN determinism test
35. WFC secondary-ordering determinism test
36. WFC idempotence test
37. WFC trace-completeness test
38. WFC ablation smoke test
39. WFC disabled/bypass audit test
40. matched-depth recomputation test
41. matched-depth settings equality test
42. matched-depth non-copying test

If any mandatory test fails, or any mandatory WFC or matched-depth self-test fails, the benchmark runner shall stop before generating manuscript-facing evidence.

---

### **31. AST Forbidden Arithmetic, WFC Bypass, and Shared-Builder Audit Contract**

The module `audit.py` shall verify:

1. forbidden arithmetic does not enter construction modules
2. construction modules do not read Excel
3. construction modules do not import `egress/gold/reporting/diagnostics`
4. Certified and Non-Certified do not share the same normative builder
5. Certified is not only a wrapper around Non-Certified
6. prohibited wording is absent from code/docstrings/reports except in rejected-implementation diagnostics

The module `audit.py` shall additionally verify:

```
strict_wfc() exists
enumerate_wfc_candidates() exists
filter_wfc_strict_candidates() exists
select_wfc_candidate() exists
κ_ZERO and κ_EVEN policies are implemented
WFC traces are emitted
canonize_btns() does not bypass strict_wfc()
```

construction modules do not accept final coefficient objects without  
WFC\_GATE status  
matched-depth recomputation is implemented  
matched-depth recomputation does not copy Certified outputs  
WFC ablation output files are generated

**The audit shall detect:**

Float constants  
float()  
Decimal()  
Fraction()  
imports from decimal  
imports from fractions  
imports from numpy  
imports from sympy  
eval()  
exec()  
numpy.poly()  
numpy.roots()  
sympy.expand()  
sympy.Poly()  
openpyxl.load\_workbook()  
pandas.read\_excel()  
xlrd  
pyxlsb  
workbook-reading APIs  
egress conversion calls from construction modules  
gold-reference calls from construction modules  
diagnostic comparator calls from construction modules  
shared Certified/Non-Certified builder patterns

**Any violation shall produce:**

FAIL  
filename  
line number when available  
violation code  
explanatory message

**New violation codes:**

WFC\_OPERATOR\_MISSING  
WFC\_CANDIDATE\_ENUMERATION\_MISSING  
WFC\_STRICT\_FILTER\_MISSING  
WFC\_NO\_LEAKAGE\_CHECK\_MISSING  
WFC\_VALUE\_PRESERVATION\_CHECK\_MISSING  
WFC\_POLICY\_SELECTION\_MISSING  
WFC\_TRACE\_MISSING  
WFC\_BYPASS\_DETECTED  
CANONIZE\_ONLY\_REPORTED\_AS\_FULL\_WFC

MATCHED\_DEPTH\_NOT\_IMPLEMENTED  
MATCHED\_DEPTH\_COPIES\_CERTIFIED\_OUTPUT  
WFC\_ABLATION\_NOT\_GENERATED

Any such violation shall block PROMOTABLE\_RUN.

---

## 32. Workbook Output Contract

For each validated degree stage  $m$ , the runner shall generate:

BTNS\_m{m:02d}\_results\_v28.xlsx

For the first stage this is:

BTNS\_m02\_results\_v28.xlsx.

Each workbook must contain at least:

- 01\_RunConfig
- 02\_InputRoots
- 03\_RootIngressBTNS
- 04\_ArithmeticSelfTests
- 05\_AdditionTests
- 06\_MultiplicationTests
- 07\_PolynomialExpansionTrace
- 08\_NonCertifiedCoefficients
- 09\_CertifiedCoefficients
- 10\_GoldComparison
- 11\_GroupSummary
- 12\_ErrorRanking
- 13\_UCCDecisionSummary
- 14\_TraceAudit
- 15\_ForbiddenArithmeticAudit
- 16\_Manifest
- 17\_SettingsSignature
- 18\_ExcelCompletenessCheck
- 19\_GoldCoefficients
- 20\_NonCertifiedError
- 21\_CertifiedError
- 22\_InputRootsAll
- 23\_InputRootsSelected
- 24\_CampaignSummary
- 25\_MethodNotes
- 26\_CertifiedPathComparison
- 27\_CertifiedDecisionChain
- 28\_CertifiedRefinementTrace
- 29\_CertifiedCandidateSelection
- 30\_WFCCConfig
- 31\_WFCOperationTrace
- 32\_WFCCandidateFamily

33\_WFCStrictAdmissibility  
34\_WFCTiePolicySelection  
35\_WFCUndefinedCases  
36\_WFCAblationConfig  
37\_WFCAblationSummary  
38\_WFCWindowSensitivity  
39\_WFCTiePolicyAblation  
40\_MatchedDepthFairness  
41\_AdaptiveAdvantageClassification  
42\_WFCAndUCCCrossSummary

The workbook must clearly distinguish:

value equality  
error comparison  
path comparison  
Certified construction decision chain  
digit legality  
strict WFC admissibility  
WFC selected canonicity  
UCC certified decision  
fixed-depth comparison  
matched-depth comparison  
adaptive advantage classification

The workbook shall not merge these concepts into one ambiguous “PASS” field.

---

### 33. CSV and JSON Sidecar Contract

The package shall produce machine-readable sidecars including:

run\_config.json  
input\_roots\_all.csv  
input\_roots\_selected.csv  
root\_ingress\_btns.csv  
arithmetic\_self\_tests.csv  
polynomial\_expansion\_trace.csv  
noncertified\_coefficients.csv  
certified\_coefficients.csv  
gold\_coefficients.csv  
noncertified\_error.csv  
certified\_error.csv  
gold\_comparison.csv  
group\_summary.csv  
campaign\_summary\_by\_k\_min.csv  
ucc\_decision\_summary.csv  
certified\_path\_comparison.csv  
certified\_decision\_chain.csv  
certified\_refinement\_trace.csv

certified\_candidate\_selection.csv  
forbidden\_arithmetic\_audit.json  
manifest.json  
settings\_signature.json  
excel\_completeness\_check.json  
method\_notes.json  
wfc\_config.json  
wfc\_operation\_trace.csv  
wfc\_candidate\_family.csv  
wfc\_strict\_admissibility.csv  
wfc\_tie\_policy\_selection.csv  
wfc\_undefined\_cases.csv  
wfc\_ablation\_config.json  
wfc\_ablation\_summary.csv  
wfc\_window\_sensitivity.csv  
wfc\_tie\_policy\_ablation.csv  
matched\_depth\_fairness.csv  
adaptive\_advantage\_classification.csv  
wfc\_ucc\_cross\_summary.csv

Each WFC operation row shall include:

operation\_id  
coefficient\_index  
stage\_index  
path\_type  
window\_width  
policy  
candidate\_count  
strict\_candidate\_count  
selected\_candidate\_id  
selected\_c\_in  
q0\_selected  
wfc\_status  
value\_preservation\_status  
no\_leakage\_status  
support\_status

---

## **34. VALID\_RUN and PROMOTABLE\_RUN Contract**

### **34.1 VALID\_RUN**

A run is VALID\_RUN if:

no forbidden arithmetic is detected  
no external Excel input is detected  
all generated BTNSNumber objects have legal digits  
all coefficient traces are complete  
all required workbooks are generated

all required sidecars are generated  
manifest is generated  
settings signature is generated  
Certified path comparison artifacts are generated  
full strict WFC implementation exists  
main campaign emits complete WFC traces  
WFC candidate enumeration is complete  
strict admissibility is checked  
no-leakage  $q_0 = 0$  is checked  
 $\kappa_{\text{ZERO}}$  /  $\kappa_{\text{EVEN}}$  selection is implemented  
matched-depth fairness control is executed  
WFC ablation artifacts are generated  
no WFC bypass is detected

A VALID\_RUN may still fail scientific promotion.

### 34.2 PROMOTABLE\_RUN

A run is PROMOTABLE\_RUN only if it is first a VALID\_RUN and additionally satisfies:

worse\_count = 0  
FALSE\_count = 0  
unresolved\_final\_TUBE\_count = 0  
Certified mean error  $\leq$  fixed-depth Non-Certified mean error  
Certified max error  $\leq$  fixed-depth Non-Certified max error  
matched\_depth\_completion = 100%  
matched\_depth\_not\_run\_invalid\_count = 0  
carry\_failure\_count = 0  
illegal\_digit\_count = 0  
trace\_completeness = 100%  
manifest reproducibility = PASS  
settings signature reproducibility = PASS  
certified\_path\_not\_distinct\_count = 0  
certified\_construction\_decision\_count > 0  
certified\_path\_distinct\_count = coefficient\_count  
wfc\_bypass\_count = 0  
wfc\_trace\_completeness = 100%  
forbidden\_arithmetic\_count = 0  
WFC ablation completeness = PASS

A strict WFC undefined state in an ablation branch shall not automatically invalidate the main campaign if it is reported as diagnostic evidence. However, a strict WFC undefined state in the main construction path must be resolved by Certified refinement/rebuild or reported as final TUBE/REJECT according to UCC. It shall not be silently accepted.

This changes promotion logic from:

good numerical result

to:

good numerical result + independent Certified path

---

### 35. Acceptance Gate for $m = 2$

The  $m = 2$  campaign passes only if all conditions below are satisfied:

```
arithmetic self-tests = PASS
digit legality checks = PASS
carry normalization checks = PASS
full strict WFC tests = PASS
WFC trace completeness = PASS
WFC no-leakage checks = PASS
WFC value-preservation checks = PASS
WFC policy-selection checks = PASS
WFC bypass audit = PASS
forbidden arithmetic audit = PASS
external Excel input audit = PASS
construction boundary lock audit = PASS
shared-builder audit = PASS
Certified wrapper audit = PASS
coefficient trace completeness = PASS
Certified path comparison = PASS
certified_path_not_distinct_count = 0
certified_construction_decision_count > 0
matched-depth fairness control = PASS
WFC ablation generation = PASS
workbook generation = PASS
CSV/JSON sidecar generation = PASS
Excel completeness check = PASS
manifest reproducibility = PASS
settings signature reproducibility = PASS
FALSE_count = 0
unresolved_final_TUBE_count = 0
worse_count = 0
Certified mean error <= fixed-depth Non-Certified mean error
Certified max error <= fixed-depth Non-Certified max error
no untraced coefficient construction
no external gold leakage into construction path
no decimal leakage through egress
no silent truncation below k_min
no discarded carry above k_max
no non-strict WFC candidate accepted as canonical
```

If any condition fails, the runner shall mark:

```
VALID_RUN
NOT_PROMOTABLE_RUN
```

or, if structural validity fails:

INVALID\_RUN  
NOT\_PROMOTABLE\_RUN

---

### 36.1. Promotion Gate for $m = 3$ to $m = 10$

A degree stage may be promoted only if the previous stage is certified as promotable.

For every degree  $m$ , acceptance conditions are:

```
worse_count = 0
FALSE_count = 0
Certified mean error <= fixed-depth Non-Certified mean error
Certified max error <= fixed-depth Non-Certified max error
illegal_digit_count = 0
carry_failure_count = 0
forbidden_arithmetic_count = 0
external_excel_input_count = 0
full strict WFC tests = PASS
WFC trace completeness = 100%
WFC bypass count = 0
WFC ablation generation = PASS
matched-depth fairness control = PASS
matched_depth_completion = 100%
matched_depth_not_run_invalid_count = 0
no non-strict WFC candidate accepted as canonical
certified_path_not_distinct_count = 0
certified_construction_decision_count > 0
trace_completeness = 100%
manifest hash reproducibility = PASS
settings signature reproducibility = PASS
Excel completeness = PASS
```

### 36.2. WFC Ablation Contract

The package shall include an explicit WFC ablation campaign.

The ablation shall vary at least:

WFC enabled versus WFC-disabled/canonize-only baseline

RN\_ZERO versus RN\_EVEN

at least three window sizes

main WFC configuration versus alternative WFC configurations

The WFC-disabled or canonize-only baseline shall be clearly labeled:

CANONIZE\_ONLY\_BASELINE

and shall not be described as full WFC.

For every ablation branch, the package shall report:

branch\_id  
wfc\_enabled  
window\_width  
policy  
candidate\_count\_total  
strict\_candidate\_count\_total  
strict\_empty\_count  
tie\_resolved\_count  
undefined\_count  
TRUE\_count  
TUBE\_count  
FALSE\_count  
IMPROVED\_count  
EQUAL\_count  
WORSE\_count  
matched\_depth\_EQUAL\_count  
matched\_depth\_DIFFERENT\_count  
final\_depth\_distribution  
runtime\_or\_operation\_count\_optional

**Required ablation branches:**

A0\_CANONIZE\_ONLY\_BASELINE  
A1\_FULL\_WFC\_RN\_ZERO\_MAIN\_WINDOW  
A2\_FULL\_WFC\_RN\_EVEN\_MAIN\_WINDOW  
A3\_FULL\_WFC\_SMALLER\_WINDOW  
A4\_FULL\_WFC\_LARGER\_WINDOW

If the main campaign uses RN\_ZERO, then RN\_EVEN must be included as an ablation branch. If the main campaign uses RN\_EVEN, then RN\_ZERO must be included as an ablation branch.

The WFC ablation is required for Q1-level credibility because it tests whether the formal WFC machinery materially changes candidate selection, canonical representatives, UCC status, adaptive rebuild behavior, final-depth distribution, or matched-depth interpretation.

---

## **37. Controlled Vocabularies**

### **37.1 Decision Tokens**

TRUE  
TUBE  
FALSE

### **37.2 Comparison Tokens**

IMPROVED  
EQUAL  
WORSE

### **37.3 Certified Construction Tokens**

CERT\_ACCEPT  
CERT\_REFINE  
CERT\_REFINE\_REQUEST  
CERT\_REBUILD  
CERT\_TUBE\_HOLD  
CERT\_REJECT

### **37.4 Certified Path Tokens**

CERT\_PATH\_DISTINCT\_FROM\_NONCERT  
CERT\_PATH\_IDENTICAL\_TO\_NONCERT\_INVALID

### **37.5 Certified Value Tokens**

CERT\_VALUE\_EQUAL\_TO\_NONCERT  
CERT\_VALUE\_DIFFERENT\_FROM\_NONCERT

### **37.6 Certified Failure Tokens**

CERTIFIED\_PATH\_NOT\_DISTINCT  
CERTIFIED\_ONLY\_POST\_CONSTRUCTION  
CERTIFIED\_WRAPPER\_DETECTED  
SHARED\_BUILDER\_DETECTED  
NO\_CERTIFIED\_CONSTRUCTION\_DECISION

### **37.7 Trace Comparison Tokens**

VALUE\_IDENTICAL\_TRACE\_DIFFERENT  
VALUE\_DIFFERENT\_TRACE\_DIFFERENT  
VALUE\_IDENTICAL\_TRACE\_IDENTICAL\_INVALID  
VALUE\_DIFFERENT\_TRACE\_IDENTICAL\_INVALID

### **37.8 Run Status Tokens**

VALID\_RUN  
INVALID\_RUN  
PROMOTABLE\_RUN  
NOT\_PROMOTABLE\_RUN

### **37.9 Gate Tokens**

PASS  
FAIL  
WARNING  
BLOCKED

### **37.10 WFC Status Tokens**

FULL\_STRICT\_WFC\_EXECUTED  
WFC\_NOT\_APPLICABLE  
WFC\_STRICT\_VALID\_UNIQUE  
WFC\_TIE\_RESOLVED\_RN\_ZERO  
WFC\_TIE\_RESOLVED\_RN\_EVEN  
WFC\_UNDEFINED\_STRICT\_EMPTY

WFC\_VALUE\_PRESERVATION\_FAIL  
WFC\_NO\_LEAKAGE\_FAIL  
WFC\_SUPPORT\_FAIL  
WFC\_TRACE\_INCOMPLETE  
WFC\_BYPASS\_DETECTED  
CANONIZE\_ONLY\_BASELINE

### 37.11 WFC Ablation Tokens

WFC\_ABLATION\_BRANCH\_COMPLETE  
WFC\_ABLATION\_BRANCH\_FAILED  
WFC\_ABLATION\_NOT\_RUN\_INVALID  
WFC\_WINDOW\_SENSITIVITY\_DETECTED  
WFC\_POLICY\_SENSITIVITY\_DETECTED  
WFC\_NO\_OBSERVED\_EFFECT

### 37.12 Matched-Depth Tokens

MATCHED\_DEPTH\_EQUAL  
MATCHED\_DEPTH\_DIFFERENT  
MATCHED\_DEPTH\_NOT\_RUN\_INVALID  
PRECISION\_ADVANTAGE\_ONLY  
NO\_ADAPTIVE\_REBUILD\_NEEDED  
CERT\_ADAPTIVE\_ADVANTAGE\_CONFIRMED  
CERT\_ADAPTIVE\_WORSE\_THAN\_MATCHED\_NONCERT  
MATCHED\_NONCERT\_REPRODUCES\_CERT  
MATCHED\_NONCERT\_DIFFERS\_FROM\_CERT

---

## 38. Prohibited Wording Clause

The following wording shall be prohibited in source comments, docstrings, README files, and reports unless explicitly used to describe a rejected implementation:

Certified path preparation only  
post-construction certified interval governance only  
same recurrence with certified=True  
only marks the trace as Certified  
preserving the same BTNS-native recurrence  
Certified wrapper  
Certified relabeling

The Certified implementation may include post-construction reports, but the code must not describe Certified as merely a preparation, wrapper, or post-construction classification layer.

This clause directly removes the dangerous wording identified in the current implementation.

---

## 39. Traceability Contract

Every Certified coefficient shall carry:

certified\_path\_id  
certified\_stage\_id  
certified\_decision\_chain  
operation\_certificate\_ids  
wfc\_trace\_ids  
candidate\_count  
accepted\_candidate\_id  
refinement\_depth\_used  
matched\_depth\_comparison\_id

No final Certified coefficient may exist without a complete Certified decision chain, complete WFC trace linkage where applicable, and matched-depth comparison linkage.

A Certified coefficient lacking a construction-time Certified decision is invalid.

Every WFC-canonical object shall carry:

wfc\_trace\_id  
wfc\_config\_id  
wfc\_candidate\_family\_id  
selected\_candidate\_id  
selected\_c\_in  
wfc\_policy  
wfc\_window\_width  
strict\_candidate\_count  
wfc\_status

No final coefficient may be used in manuscript-facing evidence without a complete arithmetic, WFC, and construction trace.

---

## 40. Reproducibility Contract

The runner shall generate:

manifest.json  
settings\_signature.json

The manifest shall include:

contract\_version  
package\_name  
reference\_label  
package\_version  
run\_id  
script\_hashes  
input\_hashes  
config\_hash  
output\_hashes  
python\_version  
platform\_summary  
timestamp\_utc

certified\_path\_policy  
certified\_candidate\_policy  
certified\_refinement\_policy  
wfc\_execution\_policy  
wfc\_window\_width  
wfc\_window\_grid  
wfc\_policy  
wfc\_boundary\_mode  
wfc\_ablation\_policy  
wfc\_ablation\_branch\_count  
matched\_depth\_fairness\_policy  
matched\_depth\_completion\_hash  
wfc\_trace\_hash  
wfc\_candidate\_family\_hash  
wfc\_ablation\_hash

The settings signature shall include:

m  
k\_min\_grid  
rounding\_policy  
wfc\_policy  
wfc\_window\_width  
wfc\_window\_grid  
wfc\_boundary\_mode  
certified\_policy  
tail\_bound\_policy  
root\_group\_count  
operation\_mode  
audit\_mode  
excel\_input\_policy  
construction\_boundary\_policy  
certified\_path\_independence\_policy  
matched\_depth\_fairness\_policy  
wfc\_ablation\_policy

---

## 41. Historical Separation Clause

This package shall not claim that Excel v13, P54, PythonApplication123, PythonApplication125, PythonApplication126, or PythonApplication127 were final BTNS-native independent Certified-path engines.

The package shall classify them as follows:

Excel v13:  
    experimental Certified mini-model

P54:  
    frozen evidence and guardrail package

PythonApplication123:  
    publication-facing v13/P54 reproduction package

PythonApplication125:  
    incomplete first migration attempt toward BTNS-native construction

PythonApplication126:  
    first operational m=2 BTNS-native pipeline

PythonApplication127:  
    cleaned m=2 BTNS-native pipeline with complete v21 tests and improved reporting

PythonApplication141 / PythonApplication\_BTNS\_Native\_v28:  
BTNS-native package with full strict WFC execution, WFC ablation,  
independent Certified and Non-Certified construction paths,  
adaptive Certified rebuild, and matched-depth fairness control.

---

## 42. Required Initial File Set

### Required files:

btns\_core.py  
btns\_wfc.py  
btns\_poly.py  
btns\_certified.py  
ingress.py  
egress.py  
gold\_reference.py  
audit.py  
reporting.py  
run\_m2\_campaign.py  
run\_wfc\_ablation.py  
README.md  
sample\_roots\_m2.csv

### Required tests:

tests\_core.py  
tests\_wfc.py  
tests\_poly.py  
tests\_certified\_path.py  
tests\_matched\_depth.py  
tests\_wfc\_ablation.py  
tests\_audit.py

No  $m > 2$  runner shall be executed, reported, archived, or described as a promotable v28 result before every preceding degree stage has passed the v28 gates.

Development-only runners for  $m > 2$  are permitted before promotion only if they are clearly labeled `DEVELOPMENT_ONLY` and their outputs are excluded from manuscript-facing evidence, promotion summaries, and archival claims.

The new module `btns_wfc.py` shall contain the formal WFC implementation. It may be called by `btns_core.py`, `btns_poly.py`, and `btns_certified.py`, but it shall not call `egress`, `gold-reference`, `diagnostics`, `Excel`, `Decimal`, `Fraction`, `NumPy`, or `SymPy` arithmetic for construction decisions.

---

### 43. First Implementation Priority Under v28

The first v28 correction shall be:

`btns_wfc.py`

because the new v28 requirement is full strict WFC execution. The implementation order shall be:

1. implement and unit-test `btns_wfc.py`;
  2. integrate `strict_wfc()` into `canonize_btns()`;
  3. connect WFC traces to `btns_core.py` arithmetic outputs;
  4. integrate WFC-aware acceptance into `btns_certified.py`;
  5. integrate WFC gates into `btns_poly.py`;
  6. run  $m=2$  campaign;
  7. promote stage-by-stage only after all gates pass.
- 

### 44. Explicit Non-Goals of v28

This contract does not require the first v28 package to prove: complete  $m=3$  to  $m=10$

- beat all earlier P54 aggregate results
- guarantee Certified produces different numerical values
- prove universal superiority of Certified arithmetic
- prove hardware speed superiority
- replace all previous historical evidence
- universal superiority of BTNS over binary or decimal arithmetic
- runtime superiority over conventional numerical libraries
- hardware feasibility of ternary computers
- that WFC changes every coefficient value
- that `RN_ZERO` is universally better than `RN_EVEN`
- that any single window width is universally optimal
- intrinsic equal-depth superiority of Certified arithmetic

The first v28 goal is narrower and stricter:

Build a correct, auditable, BTNS-native  $m=2$  engine with independent Certified and Non-Certified construction paths. that executes full strict WFC, reports WFC ablation, preserves independent Certified/Non-Certified construction paths, and enforces matched-depth fairness.

---

## 45. Final Acceptance Statement

A package satisfies Script Contract v28 only if it demonstrates, for  $m = 2$ , that:

1. roots enter from decimal strings only at the ingress boundary;
2. no external Excel workbook is read as input;
3. Certified, fixed-depth Non-Certified, and matched-depth Non-Certified receive the same roots, same rounding policy, same WFC policy, same WFC window definition, and same CampaignConfig where fairness requires equality;
4. roots are converted into BTNS digit objects;
5. all coefficient-producing addition and multiplication occur through BTNS primitives;
6. no decimal, rational, floating-point, symbolic, NumPy, SymPy, or workbook-derived arithmetic enters the construction path;
7. no egress decimal value leaks back into construction;
8. no gold value leaks into construction or candidate selection;
9. full strict WFC is executed where canonization is required;
10. WFC candidate enumeration uses

$$W_w = \{0, -1, \dots, -w\}, \quad C_{\text{in}} = \{+1, 0, -1\}, \quad c_{\text{in}} \in C_{\text{in}}$$

11. strict WFC admissibility constructs and filters  $\mathcal{A}_w^{\text{strict}}$ ;
12. WFC selection uses  $\kappa_{\text{ZERO}}$  or  $\kappa_{\text{EVEN}}$ ;
13. no candidate with  $q_0 \neq 0$  is accepted as strict;
14. no WFC candidate is accepted without value-preservation or declared boundary compensation;
15. no non-strict candidate is silently promoted as canonical;
16. WFC traces are complete;
17. WFC ablation is generated and reported;
18. Non-Certified and Certified outputs are both BTNS-native;
19. Certified is not a wrapper, relabeling layer, or post-construction-only certificate over Non-Certified;
20. Certified has a distinct construction controller;
21. Certified contains construction-time decisions;
22. Certified traces differ structurally from Non-Certified traces;
23. Certified may adaptively rebuild only through declared ingress/refinement boundaries and native BTNS reconstruction;
24. identical final coefficient values are acceptable only if Certified construction traces are distinct;
25. matched-depth Non-Certified recomputation is executed for every Certified final coefficient;
26. fixed-depth and matched-depth comparisons are reported separately;
27. any Certified improvement is interpreted according to matched-depth evidence;
28. external gold is used only for evaluation/reporting;
29. all traces are complete;
30. all manuscript-relevant results appear in Excel;
31. all machine-verifiable sidecars are generated;
32. all mandatory self-tests pass;
33. all forbidden arithmetic, WFC bypass, shared-builder, and matched-depth audits pass;
34. manifest and settings signatures record WFC and matched-depth settings;
35. no silent truncation occurs below  $k_{\text{min}}$ ;
36. no carry is discarded above  $k_{\text{max}}$ ;
37. all m=2 promotion gates pass.